

Conducción Autónoma en Duckietown mediante Aprendizaje por Refuerzo profundo

Memoria técnica — Proyecto Final MSc IA

Adolfo (AdolfoPM02)

Junio 2026

Abstract

Se entrena un agente de *Deep Reinforcement Learning* para conducir en el simulador 3D **Duckietown** usando únicamente píxeles de la cámara frontal, con un enfoque de **RL puro**: la política aprende directamente de la imagen a velocidades de rueda, sin controladores, PID, avance forzado ni vocabularios de acción que codifiquen conocimiento de conducción. El trabajo cubre tres fases: (1) Q-Learning tabular *desde cero* en **FrozenLake-v1**; (2) dos baselines con Stable-Baselines3 —**DQN** (acción discreta) y **PPO** (acción continua)—; y (3) un algoritmo avanzado (**SAC**) y una depuración sistemática del *pipeline*. La contribución central de la memoria es **honest**: la mayor parte del esfuerzo fue diagnosticar y corregir una serie de fallos de infraestructura que invalidaban silenciosamente los resultados —caída silenciosa a un entorno *mock* de ruido, vídeo de render incorrecto, una convención de acción mal documentada y, sobre todo, una **doble normalización de la imagen en la CNN** que reducía las entradas ~ 255 veces. Tras corregir la normalización, un **PPO entrenado 50k pasos y afinado hasta 100k** (RL puro, `action_mode=wheels`) pasa a ser el modelo final `best_agent.zip`: mejora la recompensa media en el mapa oculto de evaluación `loop_obstacles` respecto al baseline anterior y produce varios episodios completos con recompensa positiva, aunque con **alta varianza** entre *resets*.

1 Introducción y objetivo

El objetivo es entrenar un agente capaz de mantenerse en el carril en Duckietown a partir de observaciones de imagen, y que **generalice** a un mapa no visto durante el entrenamiento. La nota depende del desempeño en el mapa oculto `Duckietown-loop_obstacles-v0`, con obstáculos estáticos.

Enfoque: RL puro. Siguiendo las diapositivas del reto, la solución entregada es aprendizaje por refuerzo *de extremo a extremo*: la red recibe la imagen y produce directamente la acción de control (velocidades de rueda). **No** se entrega ningún controlador clásico, PID, heurística de avance forzado ni *action mode* diseñado para conducir (p.ej. maniobras discretas predefinidas o velocidades acotadas a mano). Esas variantes se exploraron como diagnóstico y se documentan como *descartadas* por inyectar conocimiento de conducción (Sección 7).

Contrato de evaluación. El modelo debe (i) esperar observaciones de forma `(1, 64, 64)` apiladas en 4 frames $\rightarrow$ `(4, 64, 64)`; (ii) llevar definidas e importables las clases personalizadas (`CustomCNN`, `wrappers`); y (iii) cargar “a la primera” en un entorno limpio reconstruido desde `requirements.txt`. Entrenar en el mapa oculto supone descalificación: está bloqueado por `ValueError` en tres puntos del código (`make_env`, `resolve_maps`, lanzador).

2 Fase 1: Q-Learning tabular (FrozenLake-v1)

Como calentamiento se implementa Q-Learning tabular sin librerías de Deep RL sobre FrozenLake-v1 8×8 *resbaladiza*. La tabla $Q \in \mathbb{R}^{64 \times 4}$ se actualiza con la ecuación de Bellman (TD(0)):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (1)$$

con política ε -greedy y decaimiento lineal de ε ($1.0 \rightarrow 0.05$), $\alpha = 0.1$, $\gamma = 0.99$ y 50 000 episodios. La política greedy aprendida alcanza una **tasa de éxito** ≈ 0.52 (frente a ≈ 0 del azar en este entorno resbaladizo). La Figura 1 muestra la curva de media móvil de recompensa. Implementación en `q_learning_sandbox.py`.

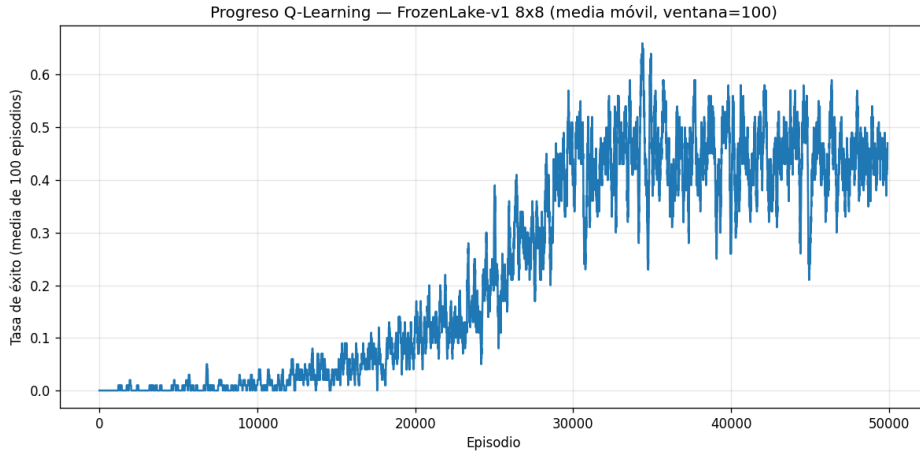


Figure 1: Fase 1: media móvil (ventana 100) de la recompensa por episodio en FrozenLake-v1 8×8 resbaladiza. El aprendizaje despegue hacia el episodio $\sim 20k$ y se estabiliza en torno a 0.45–0.50.

3 Entorno, pipeline de visión y arquitectura

Entorno. Duckietown se ejecuta sobre la API de *gym clásico* (0.25.2), no Gymnasium, por compatibilidad con *gym-duckietown* (daffy, versión 6.1.34). *DuckieWrapper* (`src/wrappers.py`) adapta esa API antigua (4-tupla) a Gymnasium (5-tupla) que espera Stable-Baselines3, y expone un espacio de acción *continuo*. La acción real del simulador son las **velocidades de las dos ruedas** $[v_{izq}, v_{der}] \in [-1, 1]^2$ (`action_mode=wheels`); el agente las produce directamente.

Pipeline de observación. El procesamiento de cada observación es:

RGB (480×640×3) → recorte 50% superior (cielo) → grises → resize 64×64 → (1, 64, 64) `uint8`

y, tras `VecFrameStack(n_stack=4)` (`src/envs.py`), (4, 64, 64). El apilado de 4 frames da al agente percepción de velocidad y giro.

CustomCNN. Extractor de características estilo *NatureCNN* (`src/cnn.py`): tres capas convolucionales $\text{Conv}(32, 8, 4) \rightarrow \text{Conv}(64, 4, 2) \rightarrow \text{Conv}(64, 3, 1)$ con ReLU, *flatten* y capa lineal a 256 características. **Importante:** Stable-Baselines3 ya normaliza las imágenes `uint8` a $[0, 1]$ (`normalize_images` por defecto) *antes* de invocar el extractor, por lo que **CustomCNN no** vuelve a dividir entre 255 (véase la Sección 6). Se conecta a SB3 vía `policy_kwargs` y es común a los tres algoritmos.

4 Algoritmos

DQN (baseline discreto). Q-Learning profundo *off-policy*. Como Duckietown es continuo, `DiscreteActionWrapper` expone 5 acciones discretas mapeadas a comandos de control. Config.: `lr=10-4`, `buffer=50k`, `learning_starts=10k`, `batch=32`, `train_freq=4`, `target_update=1000`, exploración ε $1.0 \rightarrow 0.05$. Se discute en la Sección 6 una incoherencia detectada en su vocabulario de acción.

PPO (baseline continuo). Gradiente de política *on-policy* con *clipping*, estable y eficiente en muestras. Config.: `lr=3 · 10-4`, `n_steps=2048`, `batch=64`, `n_epochs=10`, $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, `clip=0.2`. Es la base del modelo final.

SAC (algoritmo avanzado, Fase 3). Actor-Crítico *off-policy* de máxima entropía, con exploración regulada automáticamente. Config.: `lr=3 · 10-4`, `buffer=50k`, `batch=256`, $\tau = 0.005$, `ent_coef=auto`. Comparte `CustomCNN` y wrappers con PPO.

5 Metodología de entrenamiento e infraestructura

El entrenamiento real se realizó en **Google Colab**. El stack de Duckietown exige **Python 3.11** (incompatible con el kernel 3.12 de Colab), por lo que se monta un **venv 3.11** y se ejecuta todo como subproceso. Se resolvieron, de forma iterativa, varios fallos nativos y de infraestructura:

- **Dependencias:** `gym-duckietown` se instala con `--no-deps` y se re-fija `numpy==1.23.5` al final, para no romper el resto del stack.
- **Render headless:** `xvfb-run + MPLBACKEND=Agg`.
- ***Segmentation fault* (Duckietown+OpenGL+SB3):** ocurría al *inicializar* el modelo con el entorno real. La solución es el orden ***model-first***: construir el modelo sobre un entorno sintético con espacios idénticos y luego `model.set_env(real)`. `train.py` y `eval.py` lo soportan con `--init-order model-first`.

El entorno real se confirma por consola: `[duckie_factory] Using real Duckietown env: Duckietown-loop_empty-v0, gym-duckietown version 6.1.34, using DuckietownEnv.`

6 Depuración: fallos que invalidaban los resultados

El hallazgo más relevante del proyecto es que varios resultados iniciales *negativos* no medían la calidad del agente, sino fallos del *pipeline*. En orden de impacto:

(1) **Caída silenciosa al *mock*.** Si `gym_duckietown` no se importaba o no registraba sus entornos, la fábrica de entornos caía *en silencio* a un `MockDuckietownEnv` que devuelve **ruido aleatorio**. Todo lo entrenado/evaluado así aprendía sobre ruido. Se corrigió `duckie_factory.py` para (i) **fallar de forma explícita** (`RuntimeError`) cuando se pide entorno real y no está disponible —nunca cae al *mock* con `use_mock=False`— y (ii) **importar `gym_duckietown` antes de `gym.make`** (ese import es lo que registra los IDs).

(2) **Vídeo de render incorrecto.** `render()` devolvía un framebuffer ruidoso bajo Xvfb y esta versión de `gym_duckietown` no expone `render_obs()`. La fuente fiable de imagen es la **observación RGB cruda** del simulador, que `DuckieWrapper` guarda en `last_rgb_frame` (`--video-source wrapper_rgb`). Esto solo afecta a la visualización, no a lo que ve el agente. Se confirma con `frame source=venv.envs[0].last_rgb_frame | shape=(480, 640, 3)`.

(3) **Convención de acción del baseline antiguo.** El baseline anterior solo conducía bien con una transformación *swap-negate* de la acción (`action_mode=wheels_fixed`): su política quedó acoplada a una convención de ruedas concreta. Para un agente *entrenado desde cero* la convención es indiferente (el agente aprende el signo que recibe recompensa); solo importa para *reutilizar* el baseline antiguo. El modelo final se entrena con `action_mode=wheels` (por defecto) y, por tanto, **no** requiere ningún truco de acción en evaluación.

(4) **Doble normalización en la CNN (crítico).** `CustomCNN.forward` dividía la entrada entre 255, pero SB3 **ya** normaliza las imágenes `uint8` a `[0,1]` con `preprocess_obs(normalize_images=True)` antes de llamar al extractor. Resultado: la CNN recibía valores en `[0, 1/255] ≈ [0, 0.004]`, unas **255 veces** más pequeños de lo debido, degradando gravemente el aprendizaje de *todos* los algoritmos. Verificado en Colab:

```
preprocess_obs max: 0.7843137... # SB3 ya normaliza a [0,1]
after extra /255 max: 0.0030757... # la CNN recibia ~255x menos senal
```

Corrección (`src/cnn.py`): eliminar la división extra, dejando `x = observations.float()`. Este fue el cambio decisivo: con la CNN corregida el entrenamiento empieza a producir episodios completos con recompensa positiva.

(5) **Vocabulario discreto incoherente en DQN.** Las acciones discretas (`config.DISCRETE_ACTIONS`) se diseñaron con semántica $[v, \delta]$ (velocidad, giro), pero `DuckieWrapper` en `action_mode=wheels` las interpreta como $[v_{izq}, v_{der}]$. Así, la acción “recto” `[0.6, 0.0]` se ejecuta como rueda izquierda 0.6 / derecha 0.0, es decir, un giro brusco: **DQN carece de una acción de avance recto real**. Por ello DQN se reporta como **baseline sospechoso/descartado**, no como modelo final.

7 Experimentos descartados (no entregables)

Por exhaustividad se probaron *action modes* que reducen la dificultad inyectando conocimiento de conducción. **Ninguno se entrega**, porque dejan de ser RL puro:

- **safe_discrete**: 5 maniobras de rueda predefinidas (recto/giros suaves). Es un *prior* de control hecho a mano.
- **v_omega_safe**: $[v, \omega]$ con v acotado a $[0.10, 0.25]$ y ω limitado. Restringe a mano la dinámica para no salirse.
- **forward_turn** (avance forzado): **no se implementó**; sería avance impuesto, no aprendido.

Igualmente se descartan controladores, PID y cualquier heurística de conducción. El entregable usa **action_mode=wheels** (acción de rueda directa, sin acotar).

8 Resultados

Todas las evaluaciones de esta sección usan la infraestructura **corregida** (entorno real con fallo explícito, `model-first`); las cifras positivas reportadas en iteraciones anteriores, previas a estos arreglos, no se reproducían y quedan *superadas*. La métrica principal es la recompensa media en el mapa oculto `loop_obstacles` (solo evaluación). La columna “mejor *rollout*” es el de mayor recompensa de un *best-of-10* cualitativo en `loop_obstacles`.

Modelo	<code>loop_obstacles</code> (media $\pm$ std / len)	mejor rollout	decisión
Baseline antiguo + <code>wheels_fixed</code>	$-869.9 \pm 548.6 / 1264$	—	fallback inicial
PPO 20k (<i>bug CNN</i>)	-924.2 ± 441.4	—	no supera baseline
DQN 20k (<i>bug CNN</i>)	-1004.6 ± 30.0	—	descartado (Secc. 6)
SAC 20k (<i>bug CNN</i>)	-1021.7 ± 81.4	—	no supera baseline
PPO 50k single-map (<i>pre-fix</i>)	$-1481.8 \pm 473.3 / 456$	—	más pasos no ayudan
PPO multimap 50k (<i>pre-fix</i>)	$-1764.9 \pm 712.8 / 460$	-887.8	multimap no ayuda
PPO cnn-fix 50k	$-2038.0 \pm 1408.0 / 1172$	1225.7	rollouts positivos, alta varianza
PPO cnn-fix 100k (ft)	$-487.6 \pm 1049.9 / 742$	1249.5	MODELO FINAL

Table 1: Evaluación en el mapa oculto `loop_obstacles` (solo evaluación, nunca entrenamiento). “cnn-fix” = con la doble normalización corregida. El modelo final (`ppo_cnnfix_loop_empty_100k_ft` $\rightarrow$ `best_agent.zip`) tiene la mejor recompensa media de la familia corregida y supera al baseline antiguo (-487.6 vs -869.9), con varios episodios completos de recompensa positiva (mejor *rollout* 1249.5). La media negativa con *std* alta refleja la elevada varianza entre *resets*.

9 Discusión

El cambio decisivo fue corregir la normalización. Antes del *fix*, ningún algoritmo (PPO/DQN/SAC, 20k o 50k, single o multimap) superaba al baseline: todos rondaban -900 a -1800 en `loop_obstacles`. Con la CNN corregida, el mismo PPO empieza a generar episodios completos con recompensa positiva (mejor *rollout* 1225.7 ya a 50k).

Por qué el modelo final es PPO cnn-fix 100k (fine-tuning). Partiendo del PPO `cnn-fix` 50k se afinaron 50k pasos adicionales (RL puro, `lr` reducido, entrenamiento solo en `loop_empty`). Sube la recompensa media en `loop_obstacles` de -2038 a -487.6 y mantiene el mejor *rollout* en torno a 1250. Es el mejor compromiso disponible: mejor media que el baseline antiguo y mejor que cualquier variante anterior al *fix*.

Alta varianza. La *std* (± 1050) y el *best-of-10* (rollouts de $+1249$ a -1397 , algunos terminando pronto) muestran que el agente conduce bien *a veces* y se sale otras según la pose inicial. Es una limitación real, no un artefacto de medida.

DQN. Se entrenó y evaluó, pero su vocabulario de acción no contiene un avance recto real (Secc. 6, punto 5); queda como baseline **sospechoso/descartado**, no como entregable.

Pureza del enfoque. El modelo final no usa `safe_discrete`, `v_omega_safe`, `forward_turn`, PID ni controladores: aprende de la imagen a velocidades de rueda directas. Esas variantes se exploraron y se descartan por inyectar conocimiento de conducción.

10 Conclusión

La contribución práctica principal fue **diagnosticar y corregir** los fallos que invalidaban silenciosamente el entrenamiento —*mock* silencioso, vídeo de render, convención de acción y, sobre todo, la **doble normalización de la CNN**— y, sobre la base ya sana, entrenar un **PPO de RL puro** (50k + fine-tuning a 100k) que supera al baseline anterior en el mapa oculto y produce conducción real en varios episodios. El modelo final entregado es `best_agent.zip` (copia de `ppo_cnnfix_loop_empty_100k_ft.zip`), evaluable sin ningún truco de acción.

Limitaciones y trabajo futuro.

- **Alta varianza** entre *resets*: el desempeño depende mucho de la pose inicial. Mitigable con más pasos y/o evaluación más amplia.
- **Presupuesto de entrenamiento limitado** (50k–100k pasos): el RL desde píxeles en Duckietown suele requerir órdenes de magnitud más. **Futuro:** entrenar $\geq 300\text{k}$ –1M pasos con **vectorización real** (SubprocVecEnv, varios entornos en paralelo) para hacerlo viable en tiempo.
- `loop_obstacles` es **más difícil** (obstáculos) y **solo de evaluación**: parte de la recompensa negativa es *distribution shift* esperado.
- **Ajuste de hiperparámetros** (p.ej. `ent_coef` > 0 para más exploración) y **revisar el vocabulario de DQN** (incluir un avance recto real, o alinear la semántica con un *action mode* coherente).
- Envolver la evaluación con **Monitor** para métricas de episodio más limpias.

Reproducibilidad

Dependencias: requirements.txt (stack exacto, stable-baselines3==2.2.1, gymnasium==0.29.1, gym==0.25.2, torch==2.12.0) + gym-duckietown aparte (--no-deps) + re-fijado de numpy==1.23.5 (detalle en COLAB_SETUP.md). Protocolo y tabla de experimentos en EXPERIMENTS.md. El modelo final se entrenó con action_mode=wheels y se evalúa *sin* --action-mode.

```
# Modelo final: PPO cnn-fix 50k + fine-tuning a 100k (RL puro, action_mode=wheels)
python train.py --algo ppo --map Duckietown-loop_empty-v0 --timesteps 50000 \
    --output ppo_cnnfix_loop_empty_50k --init-order model-first
python train.py --algo ppo --map Duckietown-loop_empty-v0 --timesteps 50000 \
    --output ppo_cnnfix_loop_empty_100k_ft \
    --init-model models/ppo_cnnfix_loop_empty_50k --learning-rate-override 5e-5 \
    --init-order model-first

# Evaluacion del modelo final en el mapa oculto (SOLO evaluacion)
python eval.py --algo ppo --model best_agent.zip \
    --map Duckietown-loop_obstacles-v0 --allow-eval \
    --init-order model-first --device cpu

# Video (camara real del robot)
python scripts/make_eval_video.py --algo ppo --model best_agent.zip \
    --map Duckietown-loop_obstacles-v0 --allow-eval \
    --video-source wrapper_rgb --rollouts 10 --init-order model-first
```